

KTPO: K-Step Test-Time Policy Optimization for Long-Horizon Discovery

Dacheng Li[†], Fangzhou Zhao[†], Shiyi Cao[†], Hao Zhang[‡], Joseph E. Gonzalez[†], Ion Stoica[†]

[†]University of California, Berkeley [‡]University of California, San Diego

Abstract

Large Language Models (LLMs) are emerging as a promising approach for discovery problems in mathematics and systems optimization. Existing search algorithms keep models frozen, yet applying reinforcement learning (RL) over full search trajectories spanning hundreds of refinement steps remains difficult. We present KTPO (K-Step Test-Time Policy Optimization), which addresses this challenge by training on short trajectories of k refinement steps. To retain the benefits of longer search, each trajectory starts from a previously discovered program rather than from scratch. In particular, KTPO selects programs near the model’s current capability frontier, estimated by the model’s improvement rate over existing programs. KTPO with 4B-9B models achieves strong performance across seven mathematical and systems optimization tasks. On mathematical problems, KTPO outperforms or approaches the human record on Circle Packing and the Erdős minimum overlap problem. On systems optimization tasks from ADRS, a benchmark suite for real-world systems optimization, KTPO outperforms human record on maximizing SQL query prefix hit rate, and surpasses the prior AI records on minimizing multi-cloud data transfer cost and on balancing expert-parallel load across GPUs. Code is available at <https://github.com/DachengLi1/KTPO>.

1 Introduction

Large Language Models (LLMs) are emerging as a promising approach for discovery problems. These tasks range from mathematical challenges like circle packing (Stephenson, 2003) and the Erdős minimum overlap problem (Haugland, 2016), to large-scale systems engineering problems such as maximizing the prefix hit rate of SQL queries and minimizing multi-cloud data transfer costs (Wooders et al., 2024; Cheng et al., 2025; Liu et al., 2025).

Existing LLM-based methods for discovery remain largely training-free (Novikov et al., 2025; Cemri et al., 2026; Lange et al., 2025; Agrawal et al., 2025). These methods find good programs by refining an initial program over hundreds of steps under evolutionary search algorithms. While reinforcement learning (RL) effectively optimizes LLMs for multi-step tasks (e.g., agents), applying it to the full search trajectories of hundreds of refinement steps remains challenging (Cao et al., 2025; Fu et al., 2025; Tan et al., 2025). Prior work tackles this challenge by applying RL updates over single-step trajectories after each search iteration (Wang et al., 2025b). While the model effectively learns to perform a single step refinement, it does not learn to explore multi-step strategies needed for complex solution spaces (Figure 3).

This paper presents KTPO (K-Step Test-Time Policy Optimization), which optimizes the LLM over short k -step refinement trajectories instead of full search trajectories spanning hundreds of steps. To retain the benefit of refinement over hundreds of steps under this shorter refinement window, KTPO starts each trajectory from a pool of previously discovered programs. The pool is then constantly updated by tracking the improvement rate of existing programs and evicting weak ones. Concretely, KTPO iterates in three phases: (1) *Refine*: sample parent programs from the pool and generate improved child programs through k -step sequential refinements with evaluator feedback; (2) *Train*: update the policy

Task	Human Record	AI Record	KTPO
<i>Mathematical discovery</i>			
Circle Packing ($\uparrow$)	2.634	2.63598 (Wang et al., 2025b) R1-0528-Qwen3-8B	2.63598 Qwen3.5-9B
Erdős Overlap ($\downarrow$)	0.38093	0.38088 (Yuksekgonul et al., 2026) gpt-oss-120B	0.38094 Qwen3-4B
<i>Systems optimization (Cheng et al., 2025)</i>			
SQL prefix hit rate ($\uparrow$)	0.692	0.775 (Cemri et al., 2026) GPT-5	0.738 Qwen3-4B
Multi-cloud data transfer ($\downarrow$)	626.2	637.1 (Cemri et al., 2026) Gemini-3-Pro	626.9 Qwen3.5-9B
Expert-parallel load balancing ($\uparrow$)	0.1265	0.1453 (Cemri et al., 2026) GPT-5 / Gemini-3-Pro	0.14638 Qwen3.5-9B

Table 1: **KTPO achieves strong performance across mathematical discovery and systems optimizations using 4B–9B models.** It outperforms or approaches human record on Circle Packing (n=26) and the Erdős Overlap minimization problem. On the system optimization benchmark ADRS (Cheng et al., 2025), it outperforms human record on maximizing SQL query prefix hit rate and sets a new AI record on minimizing multi-cloud data transfer cost. It also sets a new AI record on balancing expert-parallel load across GPUs (EPLB).

with GRPO using improvement-based rewards (Shao et al., 2024); (3) *Update pool*: record the fraction of child programs that outperform their parent programs, estimate an eviction threshold, and evict weak parent programs (Figure 1, details in Section 3).

We evaluate KTPO with open-weight 4B–9B models on seven open-ended tasks spanning mathematical discovery and systems optimization (Section 4). On Circle Packing, KTPO with Qwen3.5-9B (non-thinking) matches the best known score (Wang et al., 2025b); on Erdős minimum overlap, it approaches the human-established bound. On systems optimization tasks from ADRS (Cheng et al., 2025), KTPO closes much of the gap to methods using GPT-5 on maximizing SQL query prefix hit rate, and establishes new AI records on minimizing multi-cloud data transfer cost and on balancing expert-parallel load across GPUs (EPLB). In summary, our contributions are:

- We propose KTPO, which optimizes LLMs for discovery problems using short k -step refinements from a pool of previously discovered programs. It further tracks the improvement rate of existing programs to evict weak ones from the pool.
- We perform ablations showing that both the discovered-program pool and improvement-rate-based pool updates contribute to the performance of KTPO.
- Across seven tasks spanning mathematical discovery and systems optimization, KTPO with open-weight 4B–9B models matches the best known score on Circle Packing and approaches the human record on Erdős minimum overlap; on the ADRS systems benchmark, it surpasses the human record on SQL query prefix hit rate and sets new AI records on multi-cloud data transfer cost and expert-parallel load balancing, achieving strong results against systems based on GPT-5, Gemini-3-Pro, and 120B models.

2 Related work

LLM-guided program search for open-ended discovery. Lehman et al. (2023) showed that LLMs can serve as effective mutation operators in evolutionary program search. FunSearch further demonstrated that combining an LLM with an automated evaluator can yield new results on open mathematical problems such as cap sets and online bin packing (Romera-Paredes et al., 2024). AlphaEvolve scaled this paradigm to a broader range of scientific and engineering tasks by maintaining a population of programs refined over hundreds of iterations, using islands and MAP-Elites as the underlying evolutionary database (Novikov et al., 2025; Mouret & Clune, 2015). Several follow-up works further improved the database

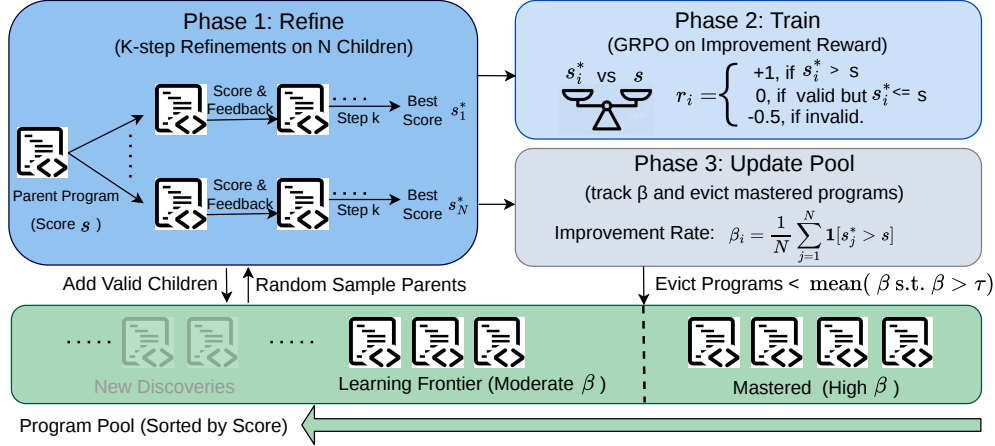


Figure 1: **KTPO optimizes the LLM using short k -step refinements from a pool of previously discovered programs, and updates this pool using program improvement rates.** Each iteration consists of three phases. (1) *Refine* (Section 3.1): sample parent programs from the pool and generate improved child programs through k -step sequential refinements with evaluator feedback. (2) *Train* (Section 3.2): assign a reward of +1 if the child improves over its parent, 0 if it is valid but non-improving, and -0.5 if it is invalid, then update π_θ via GRPO. (3) *Update pool* (Section 3.3): record the fraction of child programs that outperform their parent program (improvement rate), estimate an eviction threshold, and evict weak parent programs from the pool.

design and search procedure (Sharma, 2025; Cemri et al., 2026; Lange et al., 2025; Liu et al., 2026; Agrawal et al., 2025). Despite these advances, the language model in this line of work remains frozen. In contrast, our method trains the language model itself to perform multi-step program improvement. In addition, we do not employ an evolutionary search algorithm; instead, we maintain a simple pool of previously discovered programs and randomly sample from it to initialize short training trajectories.

Test-time training for discovery problems. Recent work has begun to study discovery problems in the test-time training regime, where the model is updated while searching for better solutions. The main challenge is horizon: strong-performing discovery systems often require hundreds of refinement steps (Sharma, 2025), making direct reinforcement learning over full refinement trajectories difficult. ThetaEvolve (Wang et al., 2025b) addresses this by applying GRPO updates to one-step refinement trajectories collected after each evolutionary search iteration, while retaining an island-based evolutionary search infrastructure with MAP-Elites and an elite archive. Concurrent work TTT-Discover (Yuksekgonul et al., 2026) also studies test-time training for open-ended optimization, introducing a PUCT-inspired rule reuse mechanism and an entropic reward tailored to discovery problems (Rosin, 2011). KTPO differs from these works in three ways. First, it replaces evolutionary search with a simple pool of previously discovered programs from which training trajectories are initialized. Second, it optimizes multi-step sequential refinements, where the model learn to explore the complex problem space (Figure 3) according to evaluator scores and diagnostic feedback (Muennighoff et al., 2025; Li et al., 2025; Agrawal et al., 2025; Chen et al., 2023). Thirdly, KTPO uses an improvement-based reward which rewards children programs that outperform parent programs.

Multi-step RL for LLMs. Recent work has extended RL post-training beyond single-turn reasoning to multi-turn interactive settings. SkyRL-Agent (Cao et al., 2025), rLLM (Tan et al., 2025) and OpenTinker (Zhu & You, 2026) provide scalable infrastructure for multi-turn agent training, GiGPO (Feng et al., 2025) studies credit assignment over multi-turn trajectories, and AgentGym-RL (Xi et al., 2025) trains agents across diverse interactive environments.

However, these settings typically involve only tens of interaction steps. More recent work pushes RL toward longer-horizon agents (Chen et al., 2025; Wang et al., 2025a), but open-ended program discovery often requires hundreds of refinement steps. KTPO tackles this regime by optimizing short multi-step refinement trajectories while reusing previously discovered programs to retain the benefits of longer-horizon search.

Training with self-generated curricula. Curriculum learning emphasizes that training is most effective when examples lie near the learner’s current frontier (Bengio et al., 2009). On fixed problem sets, methods such as PRIME (Cui et al., 2025), Polaris (An et al., 2025), and RLVE (Zeng et al., 2025) adapt training difficulty based on model performance, while DAPO (Yu et al., 2025a) filters degenerate groups in which all samples succeed or fail. Similar ideas have also been extended to self-generated problems: R-Few selects generated problems according to the model’s empirical success rate, yielding a self-generated curriculum (Yu et al., 2025b). KTPO applies this idea in the discovery setting. Search-generated parent-child program pairs provide the training data, while the improvement rates of sampled parent programs determine which starting points remain in the pool.

3 Method

We consider open-ended optimization problems defined by a deterministic evaluator $s(\cdot)$ that assigns a scalar score to candidate programs; without loss of generality, we orient scores so that higher is better¹. Given a seed program p_0 and a language model policy π_θ , the goal is to find a program that maximizes $s(\cdot)$.

KTPO optimizes the LLM over short k -step refinement trajectories instead of full search trajectories spanning hundreds of steps. To retain the benefit of longer-horizon refinement under this shorter training horizon, it initializes each trajectory from a program pool $\mathcal{P}$ of previously discovered programs. The pool is updated using the improvement rates of existing programs, so that weak programs are evicted as search progresses. Each iteration then consists of three phases (Algorithm 1): (1) *Refine*—sample parent programs from $\mathcal{P}$ and generate improved candidates through k -step sequential refinement (Section 3.1); (2) *Train*—update π_θ via GRPO using an improvement-based reward (Section 3.2); and (3) *Update pool*—update $\mathcal{P}$ using the improvement rates of existing programs (Section 3.3).

3.1 Refine: multistep sequential refinement

In the refine phase, we randomly sample B programs from $\mathcal{P}$; following standard evolutionary terminology (Novikov et al., 2025), we call these sampled programs *parents* and the variants generated from them *children*. For each parent p_i , we generate N children, each through k steps of sequential revision grounded in evaluator feedback (Li et al., 2025; Muennighoff et al., 2025; Chen et al., 2023). At each step, the model receives its previous attempt together with the evaluator’s score and diagnostic feedback (e.g., error messages and constraint violations; see Section D), and produces a revised program. This yields a sequence $c_j^{(1)}, \dots, c_j^{(k)}$, in which each refinement step conditions on the previous attempt and its feedback. In practice, $k > 1$ is important for performance (Section 4.3).

3.2 Train: GRPO on improvement reward

We use scored parent-child pairs from the refine phase to update the policy toward generating improvements over their parents. For child j of parent p_i , let $s_j^* = \max_{\ell=1}^k s(c_j^{(\ell)})$ denote the best score across its k revision steps. We take the maximum rather than the mean so that the model can explore diverse refinement strategies within a trajectory without being penalized for intermediate missteps (Section D). The reward is $r_j = +1$ if $s_j^* > s(p_i)$, and $r_j = 0$ otherwise. Additionally, Wang et al. (2025b) finds that penalizing invalid generations

¹for lower-is-better objectives, we apply a task-specific monotone transform, e.g., reciprocal scoring for Erdős minimization problem (4).

Algorithm 1 KTPO: K-Step Test-Time Policy Optimization

Require: Initial program p_0 , policy π_θ , group size N , refinement depth k , batch size B , eviction thresholds τ_0, τ_T , total iterations T , sweep window W

```

1:  $\mathcal{P} \leftarrow \{p_0\}$   $\triangleright$  Pool of discovered programs
2:  $\mathcal{R} \leftarrow []$   $\triangleright$  Rolling window of mastered-parent scores
3: for iteration  $t = 1, \dots, T$  do
4:    $\tau_t \leftarrow \tau_0 + (\tau_T - \tau_0) \cdot t/T$   $\triangleright$  Mastery threshold schedule
5:   // Phase 1: Refine
6:   for  $i = 1$  to  $B$  do
7:     Sample parent  $p_i \sim \text{Uniform}(\mathcal{P})$ 
8:     for  $j = 1$  to  $N$  do
9:        $\{c_{i,j}^{(1)}, \dots, c_{i,j}^{(k)}\} \leftarrow \text{MULTISTEPREFINE}(\pi_\theta, p_i, k)$   $\triangleright$   $k$ -step sequential refinement
10:       $s_{i,j}^* \leftarrow \max_{\ell=1}^k s(c_{i,j}^{(\ell)})$   $\triangleright$  Best score in trajectory
11:       $\mathcal{P} \leftarrow \mathcal{P} \cup \{c_{i,j}^{(\ell)} : s(c_{i,j}^{(\ell)}) > 0\}$   $\triangleright$  Add valid discovered programs
12:    end for
13:  end for
14:  // Phase 2: Train
15:  Update  $\pi_\theta$  via GRPO on all  $B$  groups with reward  $r_{i,j}$   $\triangleright \{+1, 0, -\frac{1}{2}\}$ 
16:  // Phase 3: Update Pool
17:  for  $i = 1$  to  $B$  do
18:     $\beta_i \leftarrow \frac{1}{N} \sum_{j=1}^N \mathbf{1}[s_{i,j}^* > s(p_i)]$   $\triangleright$  Improvement rate
19:    if  $\beta_i \geq \tau_t$  then
20:      Append  $s(p_i)$  to  $\mathcal{R}$   $\triangleright$  Record mastered parent score
21:    end if
22:  end for
23:   $\bar{s} \leftarrow \text{mean}(\mathcal{R}[\text{last } W \text{ entries}])$   $\triangleright$  Eviction threshold
24:   $\mathcal{P} \leftarrow \{p \in \mathcal{P} : s(p) \geq \bar{s}\}$   $\triangleright$  Evict weak programs
25: end for

```

(e.g. no code block emitted, output identical to the parent, or runtime crash) provides a stronger learning signal than treating them as zero. We therefore assign $r_j = -0.5$ to invalid children:

$$r_j = \begin{cases} +1, & \text{if } s_j^* > s(p_i), \\ 0, & \text{if valid but } s_j^* \leq s(p_i), \\ -0.5, & \text{if invalid.} \end{cases} \quad (1)$$

This trajectory-level reward is assigned to the entire k -step rollout, so all tokens across all revision steps share the same reward r_j . We apply GRPO (Shao et al., 2024), treating each parent p_i and its N children as one group (B groups per iteration). Within each group, per-child advantages are computed as $\hat{A}_j = (r_j - \bar{r})/\sigma_r$, where $\bar{r}$ and σ_r denote the group mean and standard deviation.

3.3 Update pool: improvement-rate-driven eviction

All intermediate programs with positive score from the refine phase are added to the pool. To keep the pool populated with effective starting points as it grows, we use the *improvement rate* to identify programs the policy has mastered and derive an eviction threshold for pruning weak programs.

For each parent p_i sampled during refinement, its improvement rate is

$$\beta_i = \frac{1}{N} \sum_{j=1}^N \mathbf{1}[s_{i,j}^* > s(p_i)], \quad (2)$$

the fraction of children that improve over it. A parent with $\beta_i \geq \tau_t$ is declared *mastered* and its score is recorded in a rolling buffer $\mathcal{R}$. The threshold τ_t decays linearly from τ_0 to τ_T

over training, allowing more aggressive pruning as the policy improves. At the end of each iteration, we compute an eviction threshold

$$\bar{s} = \text{mean}(\mathcal{R}[\text{last } W]),$$

the rolling mean of recent mastered-parent scores, and evict all programs with scores below $\bar{s}$. In this way, the pool tracks the policy’s learning frontier: mastered programs determine the eviction threshold, and all programs below that threshold are removed (Figure 1).

4 Experiments

We evaluate KTPO on seven open-ended optimization tasks spanning mathematical discovery and systems engineering. Our experiments demonstrate that:

1. On open mathematical problems, KTPO with open-weight 4B–9B models approaches or matches the best known scores (Section 4.2).
2. On system optimization tasks, KTPO is competitive with methods using GPT-5 and Gemini-3-Pro (Section 4.2).
3. We ablate key design choices: the program pool, improvement-rate eviction, and multi-step revision, to validate each component (Section 4.3).

4.1 Experimental setup

Tasks. We evaluate on two open mathematical problems studied by AlphaEvolve (Novikov et al., 2025) and ThetaEvolve (Wang et al., 2025b), and five systems engineering tasks from the ADRS benchmark (Cheng et al., 2025):

- **Circle Packing** ($n = 26$): Pack 26 non-overlapping circles into a unit square to maximize the sum of radii. This is a classical computational geometry problem where AlphaEvolve achieved a score of 2.63586 (Novikov et al., 2025).
- **Erdős Minimum Overlap**: A long-standing open problem in combinatorics (Haugland, 2016). Lower is better.
- **LLM-SQL**: Optimize column ordering in database schemas to improve LLM-based text-to-SQL performance (Liu et al., 2025). Higher normalized score indicates better query accuracy (Cheng et al., 2025).
- **Cloudcast**: Minimize the cost of broadcast operations in distributed systems by optimizing communication schedules (Wooders et al., 2024). Lower cost (in dollars) is better (Cheng et al., 2025).
- **EPLB, Prism, TXN**: Three further ADRS tasks: balancing expert-parallel load across GPUs for Mixture-of-Experts serving, model placement, and transaction scheduling (Cheng et al., 2025).

These tasks share a common structure: given an initial program, the goal is to iteratively produce improved programs scored by a deterministic evaluator. The initial programs are the standard ones included with each benchmark, and we did not tune them (Section C).

Baselines. For mathematical tasks, we compare against AlphaEvolve (Novikov et al., 2025) (Gemini-2.0-Flash/Pro), ShinkaEvolve (Lange et al., 2025) (ensemble of 6 frontier LLMs), ThetaEvolve (Wang et al., 2025b) (DeepSeek-R1-Distill-Qwen3-8B), and TTT-Discover (Yuksekgonul et al., 2026) (gpt-oss-120b). For system optimization tasks, we compare against OpenEvolve (Sharma, 2025), GEPA (Agrawal et al., 2025), ShinkaEvolve, and AdaEvolve (Cemri et al., 2026), using GPT-5 and Gemini-3-Pro. All baselines use models with 8B or more parameters, or proprietary APIs. To further isolate the contribution of RL training, we additionally run ThetaEvolve with the same Qwen3-4B model as an inference-only control (Section 4.3).

Model. We report results on **Qwen3-4B-Instruct-2507** (Yang et al., 2025) and **Qwen3.5-9B-NoThink** (Qwen Team, 2026), run in non-thinking mode. We perform our ablations on Qwen3-4B-Instruct-2507.

Method	Model	Circle Packing $\uparrow$	Erdős $\downarrow$
Human	—	2.634	0.380927
AlphaEvolve	Gemini-2.0	2.63586	0.380924
ShinkaEvolve	6 LLMs	2.63598	—
ThetaEvolve	Qwen3-8B	2.63598	—
TTT-Discover	gpt-oss-120B	—	0.380876
KTPO (Ours)	Qwen3-4B	2.63579	0.380938
KTPO (Ours)	Qwen3.5-9B	2.63598	0.380950

Table 2: **KTPO approaches or matches the best known scores on both mathematical discovery tasks.** With Qwen3-4B, KTPO approaches prior SOTA; with Qwen3.5-9B, it ties the best known Circle Packing score. Circle Packing: sum of radii ($\uparrow$); Erdős: upper bound on overlap constant ($\downarrow$). “—” = not evaluated. **Bold** = best per column.

Implementation details. We run KTPO with the following configuration: Each rollout generates $B = 16$ parent-child groups with $N = 16$ samples per group, yielding 256 candidate programs per rollout. We apply $k = 8$ steps of multistep sequential revision; the best score across all k steps determines the improvement reward (Section 3). Training uses GRPO (Shao et al., 2024) with the asymmetric clip-higher setting from DAPO (Yu et al., 2025a), using a clipping range of $[0.2, 0.28]$. We additionally apply truncated importance sampling (Yao et al., 2025) to correct for the difference between rollout generation and gradient updates. The adaptive eviction starts with a threshold of $\tau_0 = 0.3$, decaying to $\tau_T = 0.0$ over training, with a rolling window of $W = 10$. We run $T = 50$ rollouts per experiment.

For generation, we use temperature 1.0 with top- $p = 1.0$ and a maximum response length of 32,768 tokens. The policy is optimized with Adam ($\text{lr} = 10^{-6}$, $\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay = 0.1) with optimizer states offloaded to CPU. All experiments run on a single node with 8 NVIDIA H100 80GB GPUs, using tensor parallelism of 8 with colocated training and inference via Slime (Zhu et al., 2025). For Qwen3-4B, we report results across 3 random seeds as mean $\pm$ std, and additionally report the best score achieved across all seeds and rollouts. Qwen3.5-9B results are from a single seed and reported as best score only.

4.2 Main results

Mathematical discovery. Section 4.2 presents results on two open mathematical problems. On **Circle Packing** ($n = 26$), KTPO with Qwen3-4B achieves 2.6353 ± 0.0004 across 3 seeds (best: 2.63579), surpassing the human record (2.634). With Qwen3.5-9B, KTPO matches the best known score of 2.63598, tying ThetaEvolve (Qwen3-8B) and ShinkaEvolve (6 LLMs). On **Erdős Minimum Overlap**, KTPO with Qwen3-4B reaches 0.380938, within 1.1×10^{-5} of the human-established bound (0.380927). TTT-Discover achieves the best known result (0.380876) using a 120B model.

System optimization. Section 4.2 presents results on five systems engineering tasks from the ADRS benchmark, with baseline numbers from Cemri et al. (2026). On **LLM-SQL**, KTPO with Qwen3-4B achieves $.734 \pm .004$ (best: .738), surpassing the human baseline (0.692) and outperforming OpenEvolve, GEPA, and ShinkaEvolve with GPT-5. On **Cloudcast**, KTPO with Qwen3.5-9B achieves \$626.9, surpassing the prior algorithmic best (AdaEvolve with Gemini-3-Pro, \$637.1) and approaching the human record (\$626.2). With Qwen3-4B, KTPO achieves $\$658 \pm 9$, outperforming OpenEvolve and ShinkaEvolve with both GPT-5 and Gemini-3-Pro. On **EPLB** (load balancing for Mixture-of-Experts), KTPO with Qwen3.5-9B achieves a best score of .14638, surpassing the prior best of .1453 (AdaEvolve with both GPT-5 and Gemini-3-Pro). On **Prism** (model placement), KTPO with Qwen3.5-9B reaches 26.256, slightly below AdaEvolve’s best of 26.37, while still substantially outperforming the human baseline (21.89). On **TXN** (transaction scheduling), KTPO with Qwen3.5-9B achieves 4149, within 4.6% of AdaEvolve’s best (4348) and substantially above the human baseline (2725). With open-weight 4B–9B models, KTPO is competitive with methods using proprietary APIs across all five tasks.

Method	Model	LLM-SQL $\uparrow$	Cloudcast (\$) $\downarrow$	EPLB $\uparrow$	Prism $\uparrow$	TXN $\uparrow$
Human / SOTA	—	0.692	626.2	0.1265	21.89	2725
OpenEvolve	GPT-5	.716	729.8	.1272	26.23	4329
GEPA	GPT-5	.713	645.7	.1445	26.23	3984
ShinkaEvolve	GPT-5	.713	812.7	.1272	26.26	4329
AdaEvolve	GPT-5	.775	640.5	.1453	26.37	4348
OpenEvolve	Gemini-3-Pro	.736	667.1	—	—	—
AdaEvolve	Gemini-3-Pro	.752	637.1	.1453	26.26	4310
KTPO (Ours)	Qwen3-4B	.738	651.3	—	—	—
KTPO (Ours)	Qwen3.5-9B	.737	626.9	.14638	26.256	4149

Table 3: **KTPO with open-weight 4B–9B models is competitive with GPT-5 and Gemini-3-Pro on system optimization.** LLM-SQL, EPLB, Prism, TXN ($\uparrow$): higher is better; Cloudcast ($\downarrow$): lower is better. We report the best score across seeds (Qwen3-4B: 3 seeds; Qwen3.5-9B: 1 seed). **Bold** = best per column. Baseline numbers from [Cemri et al. \(2026\)](#).

Method	Circle Packing $\uparrow$	Cloudcast (\$) $\downarrow$
OpenEvolve	2.488	911.2
ShinkaEvolve	2.457	804.4
GEPA	2.522	825.3
AdaEvolve	2.422	798.8
KTPO (ours)	2.617	698.1

Table 4: **KTPO outperforms all four evolutionary baselines with the same model and compute budget** (Qwen3-4B, 20k evaluated programs, 2 seeds each).

Full training curves for Circle Packing, Erdős, LLM-SQL, and Cloudcast are provided in Figure 5 (Appendix).

Comparison with ThetaEvolve. Figure 2 compares KTPO and ThetaEvolve on Circle Packing under matched compute (~ 51 k LLM evaluations), the same model (Qwen3-4B), and hardware ($8 \times H100$). We run ThetaEvolve with its default configuration: diff-based edits, retaining islands, MAP-Elites, and the elite database as its suggested XL configuration. KTPO uses full-program generation with $k=8$. At this budget, KTPO reaches 2.632 ± 0.002 compared to 2.559 ± 0.045 for ThetaEvolve over 3 seeds.

Matched-compute comparison with evolutionary baselines. We further run all four evolutionary baselines (OpenEvolve, ShinkaEvolve, GEPA, AdaEvolve) with the same backbone (Qwen3-4B) and the same 20k-program budget as KTPO, on Circle Packing and Cloudcast (two seeds each). With model and compute fixed, the only variable is the method. As shown in Table 4, KTPO outperforms every baseline on both tasks under this matched setting, indicating that the gains come from the method rather than from the model or the compute budget. (KTPO’s scores here are capped to the 20k-sample budget rather than the full run reported in Section 4.2.)

4.3 Analysis

We ablate each component of KTPO to understand its contribution to the overall system. All ablations are run on Circle Packing ($n=26$) with Qwen3-4B using the same compute budget (50 rollouts, $8 \times H100$) unless otherwise noted.

Pool vs. no pool. The most fundamental structural question is whether the program pool is necessary. We compare KTPO against a variant that always starts from the initial seed program p_0 at every iteration, without sampling from a pool of discovered programs. Without the pool, the model reaches 2.624 ± 0.004 at 102k evaluations—a -0.011 drop from

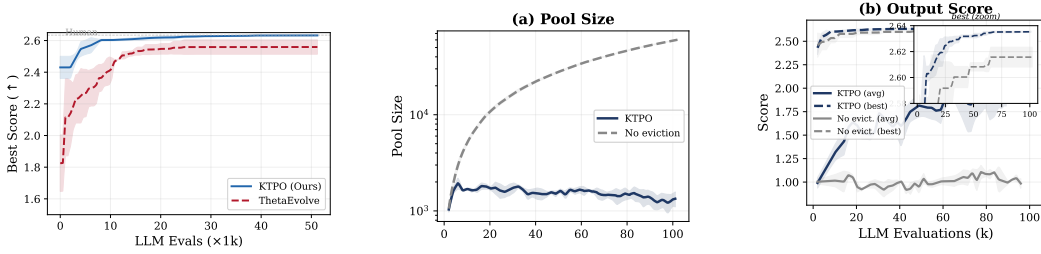


Figure 2: **Left: Comparison with ThetaEvolve under matched compute** ($\sim 51k$ LLM evaluations, Qwen3-4B, $8 \times H100$): 2.632 ± 0.002 vs. 2.559 ± 0.045 over 3 seeds. Following prior work (Sharma, 2025; Lange et al., 2025; Wang et al., 2025b), we use a tolerant evaluator. **(a), (b): Improvement-rate eviction keeps the pool compact and systematically improves generation quality.** (a) Without eviction the pool grows to 60,000+; with eviction it stays at $\sim 1,300$. (b) Solid: average best-of- k output score per batch; dashed: cumulative best.

full KTPO (2.635), confirming that building on previously discovered programs is important for continued improvement.

Improvement-rate eviction. We test whether the model’s improvement rate provides a useful signal for pool management by comparing against a variant with no eviction, where all discovered programs are kept in the pool indefinitely. Figure 2 reveals why eviction matters. Without eviction, the pool grows unboundedly to $\sim 60,000$ programs (Figure 2a), diluting sampling toward low-quality parents. Consequently, the average best-of- k output score per batch—the mean, across each batch, of the highest score achieved among k revision steps—stays flat near 1.0 throughout training (Figure 2b). With improvement-rate eviction, KTPO maintains a compact pool of $\sim 1,300$ frontier programs, and the average output score rises steadily to ~ 2.0 (Figure 2b, solid), confirming that a curated pool directly improves generation quality. This yields 2.635 vs. 2.616 without eviction (-0.019).

Multistep revision. We compare $k \in \{1, 4, 8\}$ steps of sequential revision under matched compute. At 51k LLM evaluations, $k=4$ (50 rollouts) achieves 2.635, while $k=1$ (200 rollouts) reaches 2.613—a gap of 0.022. This shows that multistep revision is critical: allowing the model to iteratively refine within a single rollout is substantially more effective than generating many independent single-step attempts. At $2 \times$ the compute, $k=4$ at 51k already matches $k=8$ at 102k (2.635 vs. 2.635), suggesting diminishing returns beyond $k=4$.

Learning vs. inference-only search. We run the full KTPO search framework—pool, improvement-rate eviction, multistep revision—but without GRPO parameter updates, so the model generates candidates using its frozen pretrained weights throughout. This isolates the contribution of RL training from the search itself. Figure 3a compares average output score per batch with and without RL training at matched compute (51k LLM evaluations, $k=1$). With training, the average output quality rises steadily (to ~ 2.2), while inference-only search stays flat (~ 0.5). This confirms that RL training consistently improves average solution quality: the model learns to produce better programs across the board, not just on occasional samples.

Wall-clock cost of training. We compare inference-only search and KTPO by wall-clock time on a matched evaluation budget (51.2k programs, the setting of Figure 3a, Circle Packing); Table 5 (left) reports the per-phase breakdown. RL incurs two additional costs over inference-only search. The first is the GRPO update itself: 5.8h, or 14% of training wall-clock. The second is indirect: as the policy is optimized, its generations become longer and more often valid, which raises the cost of both generation and program evaluation (a combined $+13.1$ h across those two phases). Together, KTPO’s wall-clock is $1.14 \times$ (update only) to $1.86 \times$ (end to end) that of inference-only search. Re-plotting the comparison of Figure 3a against wall-clock rather than evaluations (Table 5, right), training still improves average output quality steadily while inference-only search stays flat at ~ 0.5 .

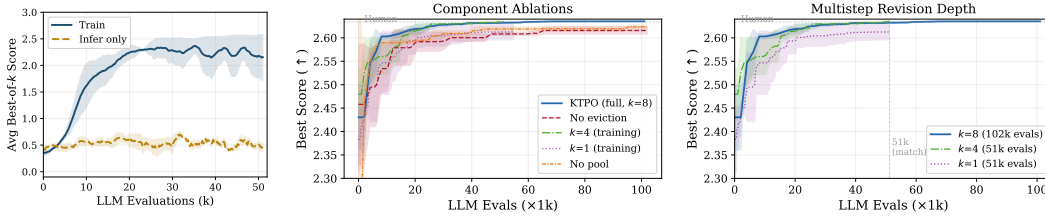


Figure 3: **RL training, improvement-rate eviction, and multistep revision each yield clear gains.** (a) Training vs. inference-only search at 51k evals: with RL training, average output quality rises steadily; without, it stays flat. (b) Component ablation at 102k evals: KTPO 2.635 ± 0.0004 ; no eviction 2.616 ± 0.008 ; no pool 2.624 ± 0.004 . (c) Revision depth k at 51k matched compute: $k=4$: 2.635 ± 0.001 ; $k=1$: 2.613 ± 0.017 . All on Circle Packing.

Phase	KTPO	Infer-only		1 h	2 h	3 h	5 h	7 h	10 h	16 h	20 h
Generation	26.3 h	19.5 h	KTPO	0.54	1.03	1.40	1.81	1.92	2.10	2.32	2.33
Program evaluation	8.7 h	2.4 h	Infer-only	0.51	0.46	0.43	0.56	0.58	0.56	0.59	0.56
GRPO weight update	5.8 h	0									
Total	40.8 h	21.9 h									

Table 5: **Wall-clock comparison of KTPO and inference-only search** on a matched evaluation budget (51.2k programs, Circle Packing). Left: per-phase wall-clock breakdown. Right: average output score versus wall-clock time; training improves steadily while inference-only search stays flat.

Visualization. Figure 4 visualizes the best solution discovered by KTPO on the circle packing ($n=26$) problem.

5 Conclusion

We presented KTPO, a policy optimization method for improving language models on discovery problems through short k -step refinement trajectories. Rather than optimizing over full search trajectories, KTPO trains the model on short refinement segments while retaining the benefits of long-horizon search through a pool of previously discovered programs. The pool is updated using program improvement rates, which keeps training focused on programs near the model’s current learning frontier. Across seven tasks spanning mathematical discovery and systems optimization, KTPO with open-weight 4B–9B models matches or surpasses systems built on much larger or proprietary models, and outperforms prior work under matched compute. Ablations further show that both the discovered-program pool and multistep sequential refinement make substantial contributions to performance.

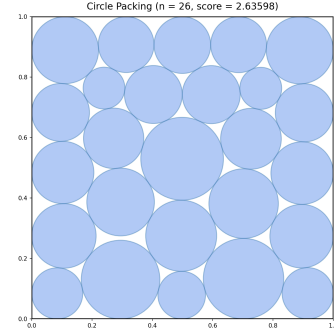


Figure 4: **Best circle packing discovered by KTPO** for $n = 26$ circles in a unit square ($\Sigma r = 2.63598$).

Acknowledgements. This work was done at the Sky Computing Lab. We sincerely thank Yiping Wang for helpful discussions, deep understanding of ThetaEvolve, and constructive design feedback. We gratefully acknowledge compute support from Lambda, Amazon, Databricks, and Anyscale. Dacheng Li and Shiyi Cao are supported by the Amazon AI PhD Fellowship.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa:

- Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- Chenxin An, Zhihui Xie, Xiaonan Li, Lei Li, Jun Zhang, Shansan Gong, Ming Zhong, Jingjing Xu, Xipeng Qiu, Mingxuan Wang, and Lingpeng Kong. POLARIS: A post-training recipe for scaling reinforcement learning on advanced reasoning models. <https://hkunlp.github.io/blog/2025/Polaris>, 2025.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pp. 41–48, 2009.
- Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, et al. Skyrl-agent: Efficient rl training for multi-turn llm agent. *arXiv preprint arXiv:2511.16108*, 2025.
- Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, et al. Adaevolve: Adaptive llm driven zeroth-order optimization. *arXiv preprint arXiv:2602.20133*, 2026.
- Kevin Chen, Marco Cusumano-Towner, Brody Huval, Aleksei Petrenko, Jackson Hamburger, Vladlen Koltun, and Philipp Krähenbühl. Reinforcement learning for long-horizon interactive llm agents. *arXiv preprint arXiv:2502.01600*, 2025.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.
- Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Bowen Wang, Alex Krentsel, Tian Xia, Mert Cemri, Jongseok Park, Shuo Yang, et al. Barbarians at the gate: How ai is upending systems research. *arXiv preprint arXiv:2510.06189*, 2025.
- Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.
- Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- Jan Kristian Haugland. The minimum overlap problem revisited. *arXiv preprint arXiv:1609.08000*, 2016.
- Jian Hu, Mingjie Liu, Shizhe Diao, Ximing Lu, Xin Dong, Pavlo Molchanov, Yejin Choi, Jan Kautz, and Yi Dong. Prorl v2: Prolonged training validates rl scaling laws. <https://hijkzzz.notion.site/prorl-v2>, August 2025.
- Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.
- Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O Stanley. Evolution through large models. In *Handbook of evolutionary machine learning*, pp. 331–366. Springer, 2023.
- Dacheng Li, Shiyi Cao, Chengkun Cao, Xiuyu Li, Shangyin Tan, Kurt Keutzer, Jiarong Xing, Joseph E Gonzalez, and Ion Stoica. S*: Test time scaling for code generation. *arXiv preprint arXiv:2502.14382*, 2025.
- Shu Liu, Asim Biswal, Amog Kamsetty, Audrey Cheng, Luis G Schroeder, Liana Patel, Shiyi Cao, Xiangxi Mo, Ion Stoica, Joseph E Gonzalez, et al. Optimizing llm queries in relational data analytics workloads. *Proceedings of Machine Learning and Systems*, 7, 2025.

- Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z Pan, et al. Evox: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*, 2026.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori B Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 20275–20321, 2025.
- Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Qwen Team. Qwen3.5: Towards native multimodal agents. <https://qwen.ai/blog?id=qwen3.5>, February 2026.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Christopher D Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>.
- Kenneth Stephenson. Circle packing: a mathematical tale. *Notices of the AMS*, 50(11): 1376–1388, 2003.
- Sijun Tan, Michael Luo, Colin Cai, Tarun Venkat, Kyle Montgomery, Aaron Hao, Tianhao Wu, Arnav Balyan, Manan Roongta, Chenguang Wang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. rLLM: A framework for post-training language agents. <https://pretty-radio-b75.notion.site/rLLM-A-Framework-for-Post-Training-Language-Agents-21b81902c146819db63cd98a54ba5f31>, 2025. Notion Blog.
- Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. UI-TARS-2 technical report: Advancing GUI agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a.
- Yiping Wang, Shao-Rong Su, Zhiyuan Zeng, Eva Xu, Liliang Ren, Xinyu Yang, Zeyi Huang, Xuehai He, Luyao Ma, Baolin Peng, et al. Thetaevolve: Test-time learning on open problems. *arXiv preprint arXiv:2511.23473*, 2025b.
- Sarah Wooders, Shu Liu, Paras Jain, Xiangxi Mo, Joseph E Gonzalez, Vincent Liu, and Ion Stoica. Cloudcast: {High-Throughput}, {Cost-Aware} overlay multicast in the cloud. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pp. 281–296, 2024.
- Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, et al. Agentgym-rl: Training llm agents for long-horizon decision making through multi-turn reinforcement learning. *arXiv preprint arXiv:2509.08755*, 2025.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your efficient rl framework secretly brings you off-policy rl training. <https://fengyao.notion.site/off-policy-rl>, August 2025.

Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025a.

Wenhao Yu, Zhenwen Liang, Chengsong Huang, Kishan Panaganti, Tianqing Fang, Haitao Mi, and Dong Yu. Guided self-evolving llms with minimal human supervision. *arXiv preprint arXiv:2512.02472*, 2025b.

Mert Yuksekogonul, Daniel Kocaja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, et al. Learning to discover at test time. *arXiv preprint arXiv:2601.16175*, 2026.

Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, et al. RLve: Scaling up reinforcement learning for language models with adaptive verifiable environments. *arXiv preprint arXiv:2511.07317*, 2025.

Siqi Zhu and Jiaxuan You. Opentinker: Separating concerns in agentic reinforcement learning. *arXiv preprint arXiv:2601.07376*, 2026.

Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2025. GitHub repository. Corresponding author: Xin Lv.

A Additional results

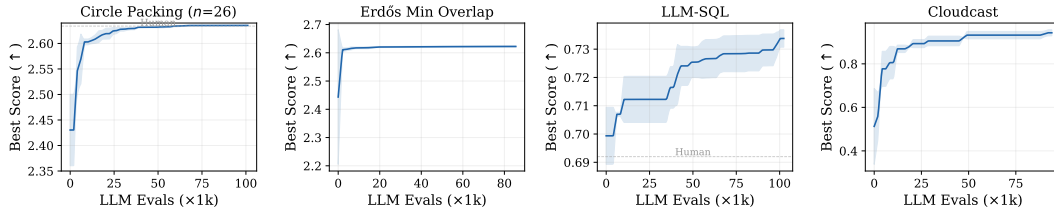


Figure 5: **KTPO improves steadily on Circle Packing, Erdős minimum overlap, LLM-SQL, and Cloudcast.** The x -axis shows cumulative LLM evaluations; the y -axis shows the best score found so far. Lines: mean over seeds; shading: ± 1 std.

B Training vs. inference-only with a smaller model

Following prior work showing that RL training improves code generation quality (Wang et al., 2025b), we additionally verify that training improves best scores in our framework using a smaller model. We run KTPO with ProRL-V2-1.5B (Hu et al., 2025) on Circle Packing ($n=26$), using diff-mode edits with $k=1$ revision step and 32×16 rollout batches, and compare KTPO with GRPO training against the same search framework with frozen weights (inference only).

Figure 7 shows the results. At matched evaluation budget ($\sim 90k$ LLM evals), KTPO with training reaches a mean best score of ~ 1.8 , compared to ~ 1.55 for inference-only search—a

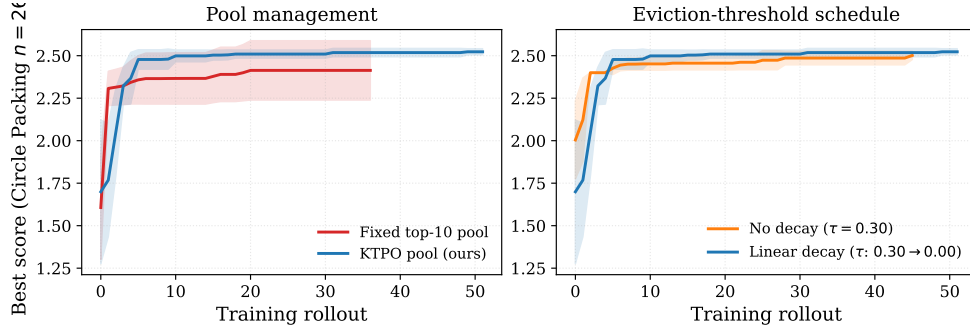


Figure 6: **Improvement-rate eviction outperforms fixed top- K pools, and the decay schedule contributes additional gains.** ProRL-V2-1.5B on Circle Packing ($n=26$), overwrite mode, $k=1$, 3 seeds; mean ± 1 std. (Left) Pool management: KTPO pool (improvement-rate eviction) reaches 2.523 ± 0.021 vs. 2.414 ± 0.215 for fixed top-10—a $10\times$ reduction in variance. (Right) Eviction-threshold schedule: linear decay ($\tau: 0.30 \rightarrow 0.00$) reaches 2.523 vs. 2.501 with fixed $\tau=0.30$.

gap of ~ 0.25 consistent with the Qwen-3 4B results in Section 4.3. The gap widens steadily with more compute, confirming that the policy continues to improve through RL and does not simply benefit from early exploration.

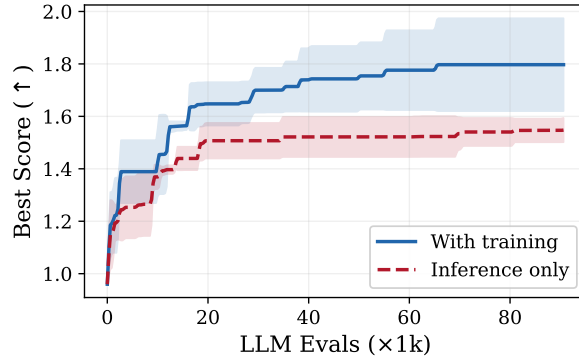


Figure 7: **Training outperforms inference-only search by ~ 0.25 even with a 1.5B model.** Circle Packing ($n=26$) with ProRL-V2-1.5B at matched eval budget. Mean ± 1 std over 3 seeds.

C Initial programs

The initial programs are the standard ones included with each benchmark; we did not tune them, and all methods are compared against the same initial program.

- **Circle Packing:** from OpenEvolve, a simple heuristic placing one large circle at the center, eight in a surrounding ring, and the rest in an outer ring: https://github.com/algoritmicsuperintelligence/openevolve/blob/main/examples/circle_packing/initial_program.py
- **Systems optimization (ADRS):** from the official ADRS implementation: <https://github.com/skydiscover-ai/skydiscover/tree/main/benchmarks/ADRS>
- **Erdős minimum overlap:** https://github.com/test-time-training/discover/blob/main/examples/erdos_min_overlap/env.py

D Prompt templates

KTPO uses a two-part prompt structure. The **system message** provides task-specific context and is sampled from a weighted list at each iteration. The **user message** includes reference programs sampled from the pool, followed by output format instructions and multi-step feedback between refinement steps. Below we show the key templates.

D.1 System message

The default system message used across all tasks:

```
You are an expert software developer tasked with iteratively improving
a codebase. Your job is to analyze the current program and suggest
improvements based on feedback from previous attempts. Focus on making
targeted changes that will increase the program's performance metrics.
```

Each task additionally samples from a task-specific context message (injected into the user prompt). For example, on Circle Packing, the context describes the packing problem, geometric insights, and the target score.

D.2 Output format instructions

When generating complete programs (rather than diffs), the model receives:

```
Output the COMPLETE, SELF-CONTAINED program as a single ```python code
block. Your code block MUST include ALL imports, ALL helper functions,
and the main entry-point function. Do NOT output partial snippets, diffs,
or incomplete code – every code block you produce will be executed as a
standalone program.
```

D.3 Reward rules

The model is explicitly told the reward structure. This prompt provides heuristic guidance to encourage improvement during generation; the actual RL training reward is the discrete signal defined in Section 3.2.

```
You will get multiple refinement attempts. After each attempt, you will
see your score and can try again. Your reward is based on the BEST
score you achieve across all attempts minus your starting score. You get
PENALIZED for producing INVALID programs (syntax errors, runtime errors, or
constraint violations). You get PENALIZED for producing the SAME score as
your previous attempt (lazy repetition). Therefore: always aim to improve,
avoid generating broken code, and do not repeat the same solution.
```

D.4 Multi-step feedback

Between refinement steps within a rollout, the model receives a structured feedback message containing: the current step number and remaining attempts, the score achieved on this step along with any domain-specific metrics (e.g., validity, boundary violations, overlaps) or error messages, the reference program score and the model's scores on all previous steps, and instructions to output a complete revised program. An example feedback message on Circle Packing after a successful step:

```
[Step 2/8, 6 remaining] this_step=2.4523 validity=True
boundary_violations=0 overlaps=3
Reference program score: 2.3891
Your scores so far:      2.4012 → 2.4523 (this step)

Think about how to improve this solution further. Output your COMPLETE
revised program as a single ```python code block.
```

When a step produces an error, the feedback includes the traceback:

```
[Step 3/8, 5 remaining] this_step=-0.1000 error=IndexError: list index
out of range
Reference program score: 2.3891
Your scores so far:      2.4012 2.4523 → -0.1000 (this step)
Think about how to improve this solution further. Output your COMPLETE
revised program as a single ```python code block.
```